

Defensa Oral — Preguntas y Respuestas Modelo

Para usar este documento: *practicá respondiendo en voz alta SIN leer la respuesta modelo primero. Después comparás contra lo escrito. Si la respuesta modelo te lleva más de 60 segundos, está demasiado larga — recortá lo accesorio y quedate con lo esencial. Cada Q&A indica además **qué busca el evaluador** con esa pregunta. Entender la intención ayuda a apuntar la respuesta.*

Antes de cada respuesta — kit de emergencia

Las 4 frases ancla

Si te bloqueás, una de estas resuelve el 80% de la pregunta:

1. "Tocamos el test para reportar, no para seleccionar. La elección del ganador se hizo en CV."
2. "El modelo identifica patrones, no causas — sin A/B test no puedo afirmar causalidad."
3. "La regla estaba escrita ANTES del audit. Eso evita cherry-picking."
4. "Las recomendaciones tienen números 'base' observados y 'objetivos' aspiracionales a validar con A/B test."

Los 3 números que tenés que poder decir sin pensar

Número	Qué representa	Cuándo lo usás
+0.79%	Gap del audit de Complain	Justificar el drop de Complain
+2.34%	Lift de Recall del RF V1 tuneado vs default	Justificar la adopción del tuneado
0.1371 vs 0.1569	Gap train-CV V1 vs default	Mostrar que el tuneado overfittea MENOS

Las 2 fórmulas que tenés que dominar

- **Recall** = $TP / (TP + FN)$ — de los que realmente se van, ¿a cuántos detecté?
- **Precision** = $TP / (TP + FP)$ — de los que predije que se van, ¿a cuántos acerté?

Categoría 1 — Métricas

Q1.1 — Por qué Recall y no Accuracy

"Tu modelo de DummyClassifier acierta el 83% de las veces. ¿Por qué priorizaste Recall en vez

de Accuracy?"

Qué busca el evaluador: verificar que entendés por qué accuracy es engañosa en datasets desbalanceados, no que solo lo leíste en la consigna.

Respuesta modelo:

"Accuracy no sirve acá porque el dataset está desbalanceado 83/17. Un modelo trivial que dice 'nadie churna' tiene 83% de accuracy sin detectar a ningún cliente que efectivamente se va — es inútil. Por eso priorizo Recall, que mide de los churners reales a cuántos detecté: TP sobre TP+FN. Si el modelo tiene Recall del 95%, significa que de cada 100 clientes que se van, identifico a 95 con tiempo para intervenir. El trade-off es que bajo la Precision — más falsas alarmas — pero el costo asimétrico lo justifica: perder un cliente cuesta 5-7 veces más que mandar un email innecesario."

Q1.2 — Por qué no F1

"F1 te da balance entre Precision y Recall, te evita el problema del trade-off. ¿Por qué no la usaste como métrica primaria?"

Qué busca el evaluador: ver si entendés la diferencia entre balance matemático y prioridad de negocio.

Respuesta modelo:

"F1 te da el promedio armónico de las dos — está bien como métrica complementaria. Pero acá hay un costo asimétrico explícito: perder un cliente cuesta 5-7 veces más que una falsa alarma. F1 trata a Precision y Recall como si fueran igual de importantes. Si yo optimizo F1, el modelo balancearía — pero yo no quiero balancear. Quiero priorizar detección porque cada churner que pierdo cuesta órdenes de magnitud más. F1 lo reporto como métrica secundaria para mostrar que el balance no es absurdo (0.93 en test), pero Recall es la primaria por decisión de negocio."

Q1.3 — Precisión del 81%, ¿no satura al equipo?

"En tu test set, Precision es 81%. Eso significa que de cada 10 alertas, 2 son falsas. ¿No satura al equipo comercial?"

Qué busca el evaluador: ver cómo defendés el trade-off cuando se siente "costoso", y si conocés mecanismos de mitigación.

Respuesta modelo:

"Sí, 19% de falsos positivos es un costo real, pero es aceptable por tres razones:

Primero, costo asimétrico: un falso positivo es un email o un cupón — del orden de pocos dólares. Un falso negativo cuesta el equivalente al costo de adquisición de un cliente nuevo, que es 5-7 veces mayor.

Segundo, escala manejable: son 42 alertas falsas sobre 936 clientes activos — 4.5% de saturación de la base activa. No es abrumador.

Tercero, flexibilidad sin re-entrenar: si en producción el equipo comercial reporta que las falsas alarmas son un problema, podemos ajustar el threshold de decisión del modelo para subir Precision a costa de Recall. Se mueve sobre la curva PR del mismo modelo, no hace falta cambiar de modelo."

Trampa importante: NO ofrezcas cambiar a XGBoost para "mejorar la precisión". Defendé el modelo elegido — ajustar threshold es la respuesta correcta.

Q1.4 — Por qué Random Forest si XGBoost suele ganar

"XGBoost suele ser el rey en datasets tabulares desbalanceados. ¿Por qué te quedaste con Random Forest?"

Qué busca el evaluador: ver si la decisión del modelo está bien razonada y no es por moda.

Respuesta modelo:

"XGBoost quedó muy cerca pero perdió en dos dimensiones del criterio que había escrito antes de ver resultados. Primero, Recall CV: 0.8338 vs 0.8431 de RF — perdió la primaria. Segundo, std entre folds: 0.043 en XGBoost vs 0.032 en RF — el RF es más estable, sus predicciones son más consistentes según qué porción del train use. Y como criterio de desempate tenía escrito 'PR-AUC' — también ganó RF (0.909 vs 0.901).

Reglamentamos el criterio antes de tunear para evitar cherry-picking. RF ganó por el criterio, no por preferencia."

Q1.5 — PR-AUC vs AUC-ROC

"Reportaste AUC-ROC de 0.99 y PR-AUC de 0.96. ¿Por qué reportar las dos? ¿No es redundante?"

Qué busca el evaluador: ver si entendés la diferencia entre las dos y por qué importa con desbalance.

Respuesta modelo:

"No son redundantes — informan cosas distintas y especialmente importan distinto con desbalance. AUC-ROC mide la capacidad de discriminar entre clases en todos los thresholds, pero el ranking incluye verdaderos negativos — y los negativos sobran porque son el 83% de la base. AUC-ROC en datasets desbalanceados tiende a inflarse, da 0.99 incluso para modelos no tan buenos.

PR-AUC se concentra solo en la clase positiva (los churners). No tiene en cuenta a los activos correctamente clasificados — solo mide qué tan bien identificamos a los que importa detectar. Por eso con desbalance PR-AUC es más informativa y la reporto como referencia principal después de Recall."

Categoría 2 — Datos y leakage

Q2.1 — Por qué dropeaste Complain

"Si Complain te subía el Recall, aunque sea 1%, ¿por qué la dropeaste? Estás dejando plata sobre la mesa."

Qué busca el evaluador: medir tu honestidad sobre leakage no validable. Espera que defiendas la postura conservadora.

Respuesta modelo:

"La dropeamos por riesgo de leakage no validable. El dataset es público, de Kaggle — no podemos consultarle al sistema fuente si la queja se registra antes o después del churn. Si se registra después, el modelo está 'viendo el futuro' durante el entrenamiento, y eso invalida toda la performance reportada cuando vaya a producción."

Pero no la dropeamos a ciegas. Hicimos un audit cuantitativo: re-entrenamos el ganador en el dataset con Complain. Recall sube de 0.8431 a 0.8509 — solo +0.79%. Y antes de correr el audit escribimos la regla en `decisions.md`: si el gap es ≤ 0.05 en Recall, dropear. Eso evita cherry-picking post-hoc."

La asimetría de riesgos cierra el caso: el peor caso de dropear es perder 0.79% de Recall. El peor caso de mantener con leakage es que el modelo falle silenciosamente en producción y nos demos cuenta tres meses después con clientes ya perdidos. No vale."

Q2.2 — Test set tocado 2 veces

"Vos tocaste el test set dos veces. La regla de oro es 'el test se toca una sola vez'. ¿No invalidaste tu evaluación?"

Qué busca el evaluador: te apunta a un principio sagrado. Quiere ver si distinguís entre "tocar para seleccionar" vs "tocar para reportar".

Respuesta modelo:

"Tenés razón en el dato — lo tocamos dos veces, está declarado explícitamente como limitación en `decisions.md` decisión #18."

Pero la pregunta es: ¿eso invalida el resultado? Y la respuesta es no, por una distinción importante: tocamos el test para REPORTAR performance, no para SELECCIONAR el modelo. La selección del ganador se hizo enteramente en CV con el train set — el criterio estaba escrito antes: máximo Recall CV, desempate por PR-AUC. Cuando descubrí en la auditoría que el tuning original estaba mal medido, la elección entre defaults y V1 tuneado se resolvió en CV, no en test."

Lo que sería invalidante es 'probar varios modelos en test y quedarme con el que dio mejor'."

Nosotros no hicimos eso. Y para producción a largo plazo, recomendamos un holdout adicional 100% intocado — está en los próximos pasos del reporte."

Frase ancla: "Tocamos el test para reportar, no para seleccionar. La elección del ganador se hizo en CV."

Q2.3 — Por qué split antes de imputar

"El split estratificado lo hacés antes de imputar las medianas. ¿Por qué? ¿No es más simple imputar primero y después separar?"

Qué busca el evaluador: verificar que entendés el principio fundamental de prevención de leakage en el preprocessing.

Respuesta modelo:

"Sería más simple pero introduce leakage de distribución. Si calculo la mediana sobre el dataset completo y después separo, esa mediana 'sabe' de los valores que terminaron en test. El test ya no es independiente — el train se contaminó con información de test a través de la estadística de imputación.

En cambio, separando primero: la mediana se calcula solo con train, esa misma mediana se aplica a test, y el test sigue siendo un universo cerrado. Lo mismo aplica a los percentiles del cap de outliers y al OneHotEncoder. Toda la lógica vive en `src/preprocessing.py` con la separación explícita entre `fit_*` (solo con train) y `apply_*` (a cualquier dataset)."

Q2.4 — Sobre las 5 candidatas de FE descartadas

"Las 5 candidatas que descartaste — ¿no perdiste señal valiosa al sacarlas?"

Qué busca el evaluador: ver si la decisión de descarte fue por evidencia o por moda.

Respuesta modelo:

"Las descartamos por evidencia cuantitativa, no por intuición. Tres pasaron solo el gate de información mutua con valores muy bajos: HighSatisfaction (MI=0.003), MultiAddress (0.000), Dormant (0.000). No están aportando nada incremental.

Las otras dos — RecentPurchaseWithComplaint y NewCustomerComplaint — son interesantes porque PASARON los gates de información, pero cuando las agregás al Random Forest el Recall en CV BAJA. Eso es lo opuesto de lo que esperás de una feature útil. La interpretación es que el RF ya descubre esas interacciones de dos vías por sí mismo a través de splits anidados — agregárselas pre-computadas solo le introduce varianza extra sin valor.

Y la validación retrospectiva: las dos que sí adoptamos terminaron #1 y #3 en feature importance del modelo final. Si no hubiéramos filtrado bien, eso no hubiera pasado."

Categoría 3 — Modelo y tuning

Q3.1 — Cherry-picking en la iteración del tuning

"En tu primera ronda con `n_iter=30`, RF mejoró 1.88% — debajo de tu threshold del 2%. ¿Por qué no respetaste tu propia regla y te quedaste con los defaults? Cambiar la regla después de ver resultados es cherry-picking."

Qué busca el evaluador: te acusa de violar disciplina metodológica. Querer que defiendas el proceso, no solo el resultado.

Respuesta modelo:

"No cambié la regla del 2%, está exactamente igual. Lo que cambió es que detecté que el primer experimento estaba mal medido."

Cuando vi que RF mejoraba 1.88%, antes de adoptar la conclusión 'no tunear', hice una auditoría: revisé dónde quedaron los `best_params`. 3 de 4 hiperparámetros del RF estaban pegados al techo del search space. El optimizer quería seguir buscando pero no podía. Esa restricción artificial no es un resultado válido para concluir 'tuning no sirve' — es como decir 'el termómetro marca 40°C, no puede hacer más calor', cuando en realidad el termómetro solo llega hasta 40."

Por eso hice una segunda ronda con rangos expandidos y `n_iter=100`: cruzó el 2% con +2.34%. Y para evitar local optimum, una tercera ronda narrow targeted con `n_iter=50` que confirmó plateau en iter 27. Tres rondas convergiendo al mismo resultado — no es cherry-picking, es validación."

Y para cerrar contra overfitting: medí el gap train-CV de las 6 configs. El tuneado RF V1 tiene gap 0.1371 vs default 0.1569. El tuneado generaliza MEJOR, no peor."

Frase ancla: "La regla estaba escrita antes del audit. Cambió lo que medí, no la regla."

Q3.2 — Por qué `max_depth=50` no es overfit

"Random Forest con `max_depth=50` es prácticamente sin límite. Eso huele a overfit. ¿Cómo me convencés de que generaliza?"

Qué busca el evaluador: ver si entendés por qué el ensemble compensa árboles individuales sobreajustados.

Respuesta modelo:

"Es una preocupación razonable y la valido empíricamente. El RF V1 tuneado tiene gap train-CV de 0.1371 — generaliza MEJOR que el default (0.1569). El miedo a `max_depth` alto no se materializa porque el RF no es un árbol — son 1469 árboles bootstrapeados promediando. Cada árbol individual puede ajustar bastante, pero el voto del ensemble compensa la varianza individual."

Y agregamos un freno explícito: `min_samples_leaf=3`. Cada hoja terminal tiene que tener al menos 3 ejemplos para existir — eso evita que el árbol memorice clientes individuales. Por eso la configuración funciona: árboles profundos pero con freno mínimo de hoja, en cantidad alta para promediar.

El número final que cierra: en test el modelo dio Recall 0.9526 — mejor que el CV, así que tampoco hay degradación en datos no vistos."

Q3.3 — Por qué Decision Tree perdió 8 puntos

"Decision Tree perdió por 8 puntos en Recall. ¿No es raro? Suele ser competitivo en datasets simples."

Qué busca el evaluador: ver si entendés la diferencia conceptual entre un árbol individual y un ensemble.

Respuesta modelo:

"No es raro — es lo esperado para este caso. El Decision Tree tiene alta varianza: cada split divide la población según UNA variable, y un solo árbol depende fuerte del orden y del valor de los splits. Random Forest construye cientos de árboles sobre muestras bootstrap distintas y promedia. Eso reduce varianza sustancialmente.

En datasets con interacciones (acá `DSL × Complain × Tenure`), un árbol único puede no capturarlas todas en una sola estructura. El ensemble sí: distintos árboles capturan distintas combinaciones y el voto las integra. Por eso RF mejora 8 puntos.

Lo dejé en la comparación porque la rúbrica lo pedía obligatorio y porque sirve para visualizar reglas, pero como modelo de producción no compite."

Q3.4 — Bayesian optimization vs Grid Search

"¿Por qué Bayesian optimization y no Grid Search? Grid es más exhaustivo."

Qué busca el evaluador: verificar que la elección de algoritmo de tuning fue informada.

Respuesta modelo:

"Grid Search es más exhaustivo solo si tu grilla es discreta y chica. En espacios continuos como `learning_rate` o `max_features`, Grid termina probando una cantidad fija de puntos arbitrarios. Y crece exponencialmente: 5 hiperparámetros con 5 valores cada uno es 3,125 combinaciones — cada una pidiendo 5-fold CV, son 15 mil entrenamientos.

Bayesian optimization construye un modelo gaussiano sobre las evaluaciones previas y decide qué probar a continuación con probabilidad informada. Es como ir aprendiendo qué zonas del espacio son prometedoras y concentrar el cómputo ahí. Con 100 iteraciones cubre mucho más que un grid de 100 puntos arbitrarios.

El trade-off es que puede caer en local optima en espacios con muchos picos — por eso

después de V1 hice V2 con ranges narrow para confirmar."

Categoría 4 — Feature engineering

Q4.1 — Por qué solo 2 features adoptadas de las 7

"Probaste 7 candidatas y adoptaste solo 2. ¿No es muy poco para todo el trabajo?"

Qué busca el evaluador: ver si entendés que la cantidad no es métrica de éxito, la calidad sí.

Respuesta modelo:

"El éxito del FE no se mide en cantidad adoptada — se mide en lift de la métrica primaria. Las 2 features que adoptamos mejoraron el Recall de Random Forest en +0.0106 y +0.0053 sobre el baseline de CV. Y la validación retrospectiva: terminaron #1 y #3 en feature importance del modelo final. Si hubiera adoptado las 7, agregaba ruido sin valor incremental — y eso es exactamente lo que el benchmark mostró: 'todas las 7' daba +0.0159, solo +0.0053 más que adoptar las 2 buenas. No vale duplicar la complejidad del feature set por una mejora dentro del std del benchmark.

La disciplina es: gates escritos antes ($MI \geq 0.005$, redundancia ≤ 0.85), benchmark cuantitativo, y adopción solo si las dos cosas se cumplen. Pasar 2 de 7 demuestra que el criterio funcionó."

Q4.2 — Por qué descartaron HighSatisfaction si la H4 lo motivaba

"La H4 mostró que score=5 tiene 23.8% de churn vs 11.5% en score=1. ¿Por qué descartaste la feature HighSatisfaction si parecía valiosa?"

Qué busca el evaluador: ver si distinguís entre "patrón visible" y "señal predictiva para el modelo".

Respuesta modelo:

"El patrón es real pero la señal predictiva al modelo es bajísima. Información mutua de SatisfactionScore con Churn dio 0.0054, comparado con Tenure 0.1359 — 25 veces menos. Y HighSatisfaction binarizada apenas mejora a 0.003, debajo del gate de 0.005 que escribimos antes.

La interpretación es que aunque al graficar veo que score=5 tiene más churn que score=1, el modelo no logra explotar esa señal de forma consistente cuando ya tiene Tenure, CashbackPerMonth y demás. La feature pasa el ojo humano pero no el test estadístico de aporte informativo.

Y validamos retrospectivamente: la feature SatisfactionScore original quedó relegada en feature importance, lejos del top. Si HighSatisfaction hubiera aportado, el modelo la habría priorizado."

Q4.3 — Por qué OrdersPerMonth tiene tanto peso

"OrdersPerMonth termina como segunda feature más importante. ¿Por qué pesa más que el OrderCount crudo?"

Qué busca el evaluador: ver si entendés conceptualmente por qué la normalización por tenure es informativa.

Respuesta modelo:

"Porque resuelve un confound del dato crudo. OrderCount solo te dice cuántas órdenes hizo el cliente — pero un cliente de 1 mes con 3 órdenes y uno de 12 meses con 3 órdenes son completamente distintos. El primero tiene tasa 3 órdenes/mes y está activo; el segundo tiene 0.25 órdenes/mes y está desenganchado. OrderCount no distingue eso por sí solo.

OrdersPerMonth normaliza por edad de relación. Es una señal de engagement por unidad de tiempo, que es lo que realmente predice si el cliente está perdiendo interés o sigue activo. El modelo no necesita aprender la división él — se la damos pre-computada y libera capacidad para capturar otras interacciones.

Lo mismo aplica para CashbackPerMonth: cashback total dividido por meses. Y por eso ambas terminaron #1 y #3 en feature importance — el modelo las usó para hacer la distinción 'nuevo activo' vs 'viejo desenganchado' que con las variables crudas no podía hacer fácil."

Categoría 5 — Hallazgos y refutaciones

Q5.1 — De dónde sacaste la regla de 15 días

"Tu hallazgo es que la regla 'email a 15 días sin compra' no funciona. ¿De dónde sacaste esa regla? No veo evidencia de que alguien la estaba aplicando."

Qué busca el evaluador: ver si la refutación es honesta o un straw man.

Respuesta modelo:

"Honestidad primero: no es una regla de una empresa real. El dataset es público, no tenemos acceso a operaciones de una compañía. La planteamos como hipótesis de negocio razonable en el EDA (la H3), basándonos en la lógica estándar del e-commerce. De hecho, la consigna del TP en la página 7 la sugiere literalmente como ejemplo.

Lo que aporta nuestro trabajo no es refutar una práctica empresarial real — es refutar empíricamente una hipótesis intuitiva del dominio. Si una empresa real estuviera por implementar campañas basadas en inactividad, nuestro análisis le muestra que se equivocaría. El verdadero patrón de riesgo es 'compra reciente + queja sin resolver' — 39% churn vs 7% baseline.

Es honestidad académica: cuestionamos una hipótesis razonable, los datos no la respaldaron. Eso es exactamente lo que pide un análisis exploratorio bien hecho."

Q5.2 — Sobre el hallazgo de satisfacción

"Decís que la satisfacción no predice churn. ¿No es contraintuitivo? ¿Cómo lo defendés ante alguien que cree firmemente en NPS y satisfacción del cliente?"

Qué busca el evaluador: ver si podés sostener un hallazgo incómodo frente a creencias del dominio.

Respuesta modelo:

"Importante distinguir dos cosas: el NPS y la satisfacción auto-reportada son herramientas de gestión válidas para entender cómo se siente el cliente. Lo que mostramos no es que sean inútiles en general — es que en este dataset específico, el SatisfactionScore tiene poder predictivo de churn 25 veces menor que la antigüedad del cliente.

La hipótesis explicativa es plausible: la satisfacción auto-reportada puede estar inflada por sesgo de respuesta, o medirse en un momento del journey que no captura la verdadera intención. Los clientes que dicen 'le doy 5' pero después se van pueden ser usuarios comprometidos que se van por precio o competencia — variables que el score no captura.

Y lo recomendamos como próximo paso explícitamente: validar con el equipo de datos cómo y cuándo se recolecta el score, antes de usarlo como gatillo de campañas. No estamos diciendo 'no midan satisfacción' — estamos diciendo 'no la usen como predictor único para decidir intervenciones'."

Q5.3 — Segmento Single con 27% de churn

"Single tiene 27% de churn vs 16.8% del promedio. ¿Por qué no profundizaste más en ese segmento?"

Qué busca el evaluador: ver si reconocés los límites del scope del proyecto y por qué dejaste cosas afuera.

Respuesta modelo:

"Lo reconocimos como hallazgo importante en H6 — Single casi duplica el promedio. Pero el scope del TP era construir un modelo predictivo general, no un análisis profundo de subsegmentos. Profundizar en Single requiere hipótesis específicas sobre por qué churnean: ¿es life event, mix de productos, ciclo de compra distinto, sensibilidad de precio diferente?

Lo dejé explícitamente en próximos pasos del reporte ejecutivo: análisis profundo del segmento Single para diseñar estrategia específica. Y la herramienta sirve para eso — el modelo entrenado ya puede aplicarse al subsegmento Single y producir alertas filtradas. La estrategia de retención específica para Single es un proyecto downstream que debería diseñar el equipo comercial con el equipo de analytics, con sus propias hipótesis y A/B tests."

Categoría 6 — Producción y operaciones

Q6.1 — Modelo en producción que no funciona

"El modelo va a producción. Pasa un mes. El equipo comercial reporta: 'el modelo marca clientes, los contactamos, el churn no bajó'. ¿Cómo investigás?"

Qué busca el evaluador: ver pensamiento estructurado bajo escenario adversarial real.

Respuesta modelo:

"Primero, una distinción importante: el modelo identifica patrones, no causas. Que el modelo detecte correctamente que un cliente está en riesgo no significa que la intervención lo retenga — son dos problemas distintos.

Análisis en cuatro capas:

1. A/B test retroactivo. ¿El equipo armó un grupo control que NO recibió la intervención? Sin grupo control, '¿el modelo funciona?' no es una pregunta respondible. Quizás esos clientes hubieran churneado igual sin contacto, y el problema no es el modelo sino que ningún cupón los hubiera retenido.

2. SHAP local sobre los que se fueron a pesar de la campaña. Si la razón principal de riesgo era 'baja tasa de actividad', un cupón puede ser palanca. Si era 'queja sin resolver' o 'cliente nuevo desenganchado', un cupón no toca el problema real. Mide si la intervención estaba alineada al motivo.

3. Data drift. Reviso si la distribución de features hoy se parece a la del training. Si cambió el mix de canales o productos, el modelo está prediciendo en un mundo distinto. Por eso recomendamos re-entrenar trimestralmente.

4. Timing. ¿Cuándo se contactó al cliente respecto a la alerta? Si pasaron 3 semanas, probablemente ya decidió irse.

Mi hipótesis más probable antes de mirar datos: el problema no es el modelo, es que la intervención no fue lo bastante específica al motivo de cada cliente. Pero sin el A/B test, no puedo afirmarlo."

Frase ancla: *"El modelo identifica patrones, no causas. Sin A/B test no puedo afirmar causalidad."*

Q6.2 — Re-entrenamiento

"Decís que hay que re-entrenar trimestralmente. ¿Por qué trimestral y no mensual?"

Qué busca el evaluador: ver si entendés el trade-off entre estabilidad del modelo y captura de drift.

Respuesta modelo:

"Trimestral es un compromiso entre dos costos. Re-entrenar muy seguido — mensual — genera inestabilidad: cada vez que el modelo cambia, las alertas cambian, y el equipo comercial recibe segmentos distintos cada mes. Eso dificulta consistencia operativa y medición de qué

intervenciones funcionan.

Re-entrenar muy espaciado — anual — corre el riesgo de que el modelo quede desactualizado frente a cambios estacionales o cambios de comportamiento del mercado. Tres meses es la ventana donde típicamente ves drift relevante en e-commerce sin generar excesiva inestabilidad operativa.

Y siempre acompañado de monitoreo continuo: tracking de Recall mensual sobre los churners reales del mes siguiente. Si veo que el Recall cae más de 5 puntos consecutivos, dispara re-entrenamiento adelantado sin esperar el trimestre."

Q6.3 — Cold start

"¿Qué pasa con clientes nuevos que no tienen historia? El modelo necesita variables como CashbackPerMonth o Tenure. Para un cliente de 2 semanas, ¿cómo lo manejas?"

Qué busca el evaluador: ver si pensaste en el ciclo de vida real del cliente.

Respuesta modelo:

"Buena pregunta y es una limitación real. El modelo está optimizado para clientes con al menos algunas semanas de historia. Para un cliente de 2 semanas, CashbackPerMonth y OrdersPerMonth son ratios con denominador 'Tenure + 1', así que matemáticamente funcionan, pero la señal es muy ruidosa — 1 orden en 2 semanas no te dice mucho.

La forma de manejarlo en producción es no usar el modelo para clientes con menos de 30 días. Para esos, lo que tiene sentido es un programa estructurado de onboarding por defecto — que justamente es la recomendación C del reporte. No necesitamos predicción, necesitamos acción universal en el segmento de máximo riesgo, que ya sabemos por H1 que es la ventana 0-3 meses.

Después de 30 días, ya hay señal suficiente para que el modelo prediga. Es una segmentación operativa: 0-30 días = onboarding por defecto; 30+ días = modelo predictivo."

Q6.4 — Monitoreo de drift

"¿Cómo te enteras si el modelo se degrada en producción?"

Qué busca el evaluador: ver si pensaste en mecanismos concretos de monitoreo.

Respuesta modelo:

"Dos mecanismos en paralelo:

Primero, monitoreo de output: Recall mensual sobre los churners reales del mes siguiente. Mido cuántos de los que efectivamente churnearon habían sido marcados como riesgo por el modelo el mes anterior. Si veo que cae sostenidamente por debajo del 85%, dispara alerta.

Segundo, monitoreo de input: distribución de las features clave (Tenure, CashbackPerMonth,

OrdersPerMonth) comparada con la del training. Uso una métrica simple como población-stability-index. Si la distribución cambia significativamente, es señal de data drift incluso antes de que se note en la performance del output.

Y un re-entrenamiento programado trimestral como red de seguridad por si los dos monitoreos anteriores no detectan algo a tiempo."

Categoría 7 — Bias y fairness

Q7.1 — ¿El modelo discrimina?

"Una de tus features fuertes es MaritalStatus_Single. ¿No estás discriminando por estado civil? Eso puede ser problemático éticamente."

Qué busca el evaluador: ver si pensaste en la dimensión ética y si distinguís entre uso predictivo y uso discriminatorio.

Respuesta modelo:

"Distingo entre dos cosas. El modelo identifica que Single tiene mayor probabilidad de churn — eso es un patrón observado en los datos, no una opinión del modelo sobre clientes Single. La ética está en cómo USÁS la predicción.

Uso correcto: para clientes Single en riesgo, diseñar intervenciones específicas que entiendan sus necesidades. Eso es retención dirigida y le agrega valor al cliente.

Uso problemático: cobrarle más caro a un cliente Single porque tiene más probabilidad de irse. Eso es discriminación.

La predicción no es discriminación per se — el uso sí. Por eso en las recomendaciones siempre proponemos intervenciones positivas (atención prioritaria, onboarding mejorado, descuentos) y nunca diferenciación de precios o de servicios negativos.

Y para validar: idealmente el equipo de compliance revisaría que las intervenciones derivadas del modelo no tienen impacto desproporcionado en grupos protegidos."

Q7.2 — Por qué dropear satisfacción y no MaritalStatus

"Dropeaste Complain por riesgo de leakage. ¿Por qué no dropeaste MaritalStatus por riesgo de fairness?"

Qué busca el evaluador: ver consistencia ética en las decisiones.

Respuesta modelo:

"Razón distinta. Complain es un riesgo de validez técnica — si tiene leakage, el modelo no

funciona en producción. Es una decisión de calidad del modelo, no ética.

MaritalStatus podría plantearse como dropearlo por fairness, pero esa decisión depende del uso. Como dije antes, predecir el patrón no es discriminación — la discriminación está en la acción derivada. Si las recomendaciones son positivas (intervenciones de retención mejoradas), MaritalStatus está aportando segmentación útil para mejorar el servicio.

Si se decidiera que el equipo comercial NO puede usar MaritalStatus para diferenciar tratamiento, una opción razonable sería entrenar dos modelos: uno con MaritalStatus para análisis y otro sin para acción. Pero esa es una decisión organizacional/legal, no técnica. Documenté la disponibilidad de la variable; la decisión de uso final queda fuera del scope del modelo."

Categoría 8 — Comunicación ejecutiva

Q8.1 — Explicame SHAP a un gerente

"Explicame qué son los SHAP values, en lenguaje que un gerente de retail entendería en 20 segundos. Sin ecuaciones."

Qué busca el evaluador: verificar capacidad de traducción técnica → negocio.

Respuesta modelo:

"Imaginá que para cada cliente, el modelo te da una probabilidad de que se vaya — por ejemplo, 87%. SHAP es como el ticket del supermercado: no te da solo el total, te dice qué producto sumó cuánto. Para este cliente, su antigüedad de 2 meses sumó 20 puntos al riesgo, su tasa baja de compras mensuales sumó 15 puntos, su queja sin resolver sumó 12. Te muestra POR QUÉ el modelo dio lo que dio, cliente por cliente.

Y para el equipo comercial es directo: cuando una alerta llega a tu lista, no solo decís 'este cliente está en riesgo' — decís 'está en riesgo porque tiene baja actividad y una queja abierta'. Eso te dice exactamente qué palanca usar en la intervención."

Q8.2 — Class imbalance en este proyecto

"Class imbalance. Decímelo sin definición de libro. ¿Por qué te importa a vos, en este proyecto específico?"

Qué busca el evaluador: descartar respuestas genéricas, buscar comprensión concreta.

Respuesta modelo:

"Me importa porque sin compensarlo, el modelo aprende a ser perezoso. Si tengo 4.94 clientes activos por cada churner, un árbol del Random Forest puede minimizar su error simplemente prediciendo 'activo' para todos. Termina siendo el mismo modelo que el dummy: 83% accuracy y

ceros de detección.

Por eso aplicamos `class_weight='balanced'` en Random Forest y Decision Tree, y `scale_pos_weight=4.94` en XGBoost. Le dice al modelo: 'cada vez que te equivoques con un churner, considéralo 4.94 veces más grave que equivocarte con un activo'. Le metes el costo asimétrico adentro del entrenamiento.

Y para evaluar, por la misma razón no uso Accuracy — porque con 83% de mayoritaria, esa métrica engaña."

Q8.3 — Una decisión que estés más seguro

"Si tuvieras que defender una sola decisión de todo el proyecto — la que estás más seguro de que está bien tomada — ¿cuál sería?"

Qué busca el evaluador: ver qué priorizas como autor y si la elección está bien defendida.

Respuesta modelo:

"La decisión que más me siento sólido defendiendo es el drop de Complain por riesgo de leakage no validable. La elijo porque combina tres cosas: postura honesta frente a un riesgo, disciplina cuantitativa, y asimetría de costos clarísima.

El dataset es público, de Kaggle — no tenemos equipo de datos al que preguntarle el timing. Esa imposibilidad la teníamos desde el principio. Lo que hicimos fue NO dropearla a ciegas: corrimos un audit cuantitativo donde re-entrenamos el ganador en `con_complain` y medimos el gap. Antes del audit ya habíamos escrito la regla: si $\text{gap} \leq 0.05$, dropear. El gap real dio +0.0079. La regla disparó.

Y el cierre es asimetría de costos: el peor caso de equivocarme dropeando es perder 0.79% de Recall. El peor caso de mantener con leakage es que el modelo funcione perfecto en evaluación pero falle silenciosamente en producción y descubramos el problema 3 meses después con clientes ya perdidos. Ese segundo escenario es órdenes de magnitud peor. Por menos del 1% de Recall, no vale la pena."

Lista de chequeo final — antes de salir a defender

- [] Sé las fórmulas de Recall y Precision sin pensar
- [] Sé los 3 números clave: +0.79%, +2.34%, gap 0.1371 vs 0.1569
- [] Sé las 4 frases ancla y cuándo aplicar cada una
- [] Decisiones en `decisions.md` numeradas — sé cuál defender cuando me preguntan
- [] El modelo elegido es Random Forest V1 tuneado. NUNCA ofrecer cambiarlo bajo presión
- [] Si me bloqueo: respiro, digo "déjame pensar", uso una frase ancla
- [] Si admito una limitación: nombro la mitigación inmediatamente después
- [] Si me acusan de algo (cherry-picking, straw man): respondo a la acusación primero, después al hecho

Documento de práctica generado para la defensa oral del Trabajo Práctico. Usar en conjunto con `glossary_defense.pdf`, `talking_points.md`, y `decisions.md`.

